Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №3

По дисциплине: «Интеллектуальный анализ данных»

Тема: «Предобучение нейронных сетей с использованием автоэнкодерного подхода»

Выполнил:
Студент 4 курса
Группы ИИ-26
Яшков П. А.

Проверил:
Никитин С. К.

Брест 2026

Цель работы:

Исследовать влияние послойного автоэнкодерного предобучения на эффективность классификации.

Общее задание

1. Обучить полносвязную сеть с числом слоёв более трёх без предобучения, оценить качество классификации.
2. Реализовать жадное послойное предобучение автоэнкодерами и последующий fine-tuning.
3. Сравнить результаты двух подходов.
4. Повторить эксперименты на датасете WDBC из ЛР №2 в дополнение к Credit Approval.
5. Оформить отчёт и исходники.

Сведения об эксперименте

Автор: Яшков П. А., группа ИИ-26, 4 курс. Вариант: 11. Все расчёты воспроизводимы: random_state/seed = 42. Исходники размещены в папке src/, результаты — src/results/.

Подготовка данных

Credit Approval: 690 заявок, 15 исходных признаков с сочетанием числовых и категориальных величин; целевой класс Approval — «+» или «−». Пропуски заполнены на обучающей части (медиана/мода), категориальные поля преобразованы OneHotEncoder. Дополнительно проведён полный эксперимент на WDBC (569 записей, 30 числовых признаков, целевой класс Diagnosis: B/M). Удалён идентификатор. Разбиение 75/25 со стратификацией, seed 42. Предобработка обучалась только на train, что предотвращает утечку данных.

Архитектура и процедура обучения

Во всех вариантах использована одна полносвязная сеть: Input → 64 → 32 → 16 → Output, с сигмоидальными активациями в трёх скрытых слоях, CrossEntropyLoss и Adam. Финальное обучение: 85 эпох. Сравнение производится по Accuracy и macro-F1 на одной тестовой выборке.

AE-предобучение: последовательные симметричные автоэнкодеры для весовых слоёв Input→64, 64→32 и 32→16. Каждый обучается 18 эпох по MSE без меток классов, после чего веса кодирующей части переносятся в соответствующий слой классификатора; затем вся сеть дообучается с метками классов.

Результаты программы

Credit Approval

Метод	Accuracy	Macro-F1	Weighted-F1	Матрица ошибок
baseline	0.8266	0.8249	0.8268	[[63, 14], [16, 80]]
autoencoder	0.8208	0.8184	0.8207	[[61, 16], [15, 81]]

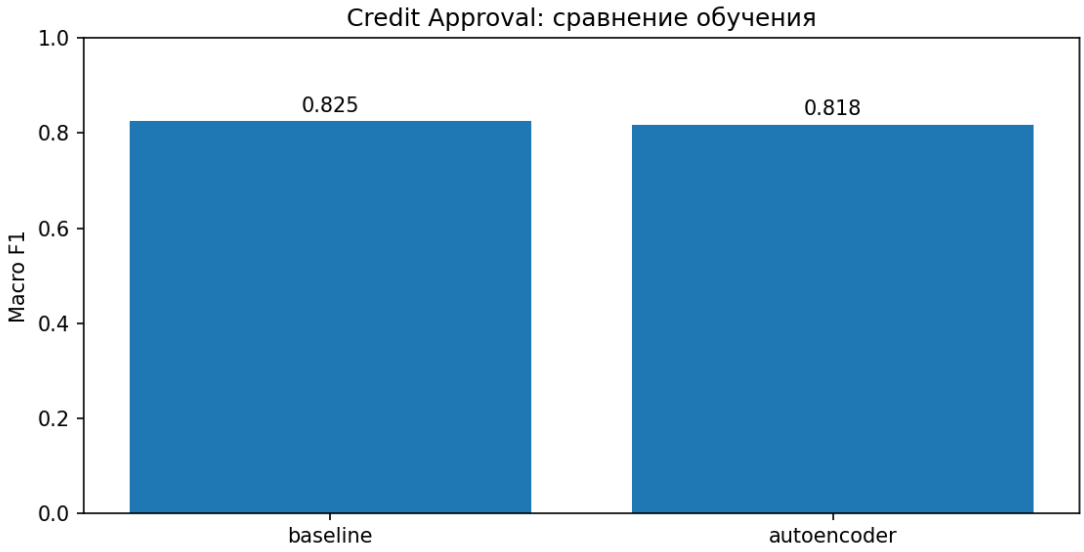
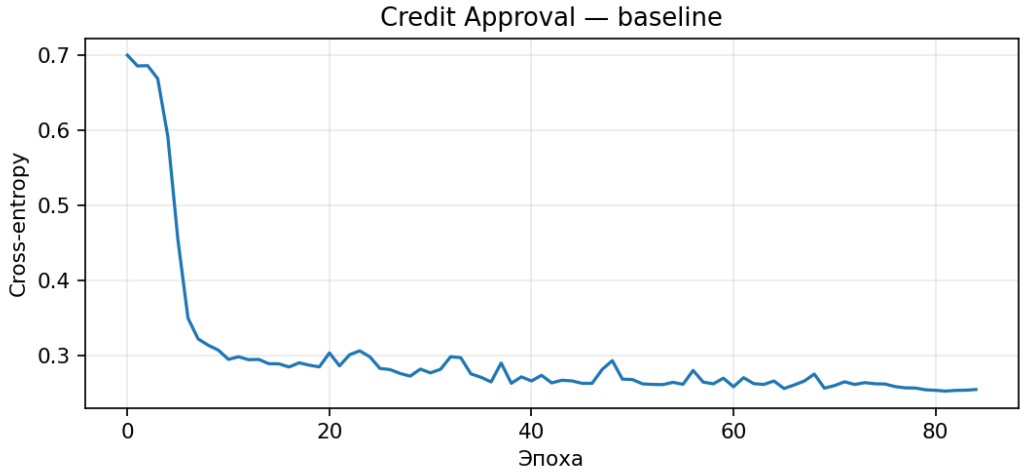
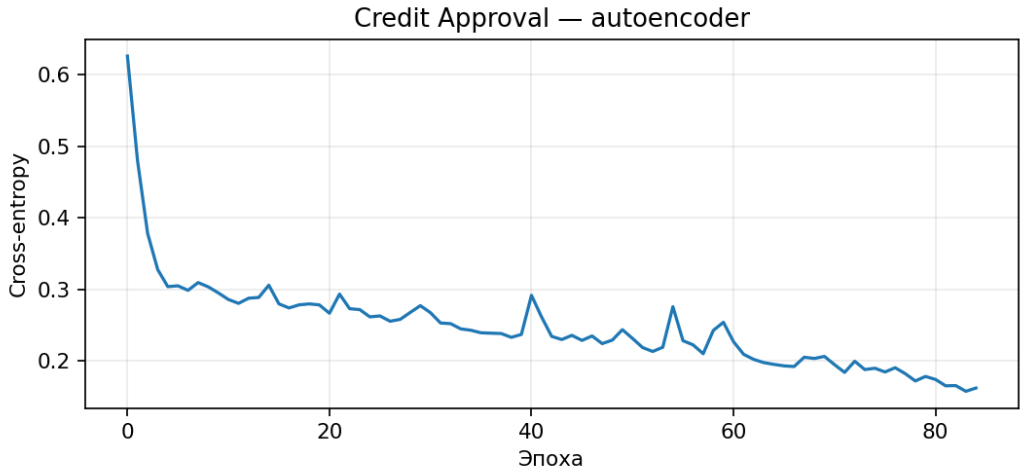


Рисунок — сравнение macro-F1 на Credit Approval.



Обучение Credit Approval: baseline, динамика CrossEntropy.



Обучение Credit Approval: autoencoder, динамика CrossEntropy.

WDBC

Метод	Accuracy	Macro-F1	Weighted-F1	Матрица ошибок

baseline	0.9650	0.9621	0.9648	[[89, 1], [4, 49]]
autoencoder	0.9720	0.9695	0.9718	[[90, 0], [4, 49]]

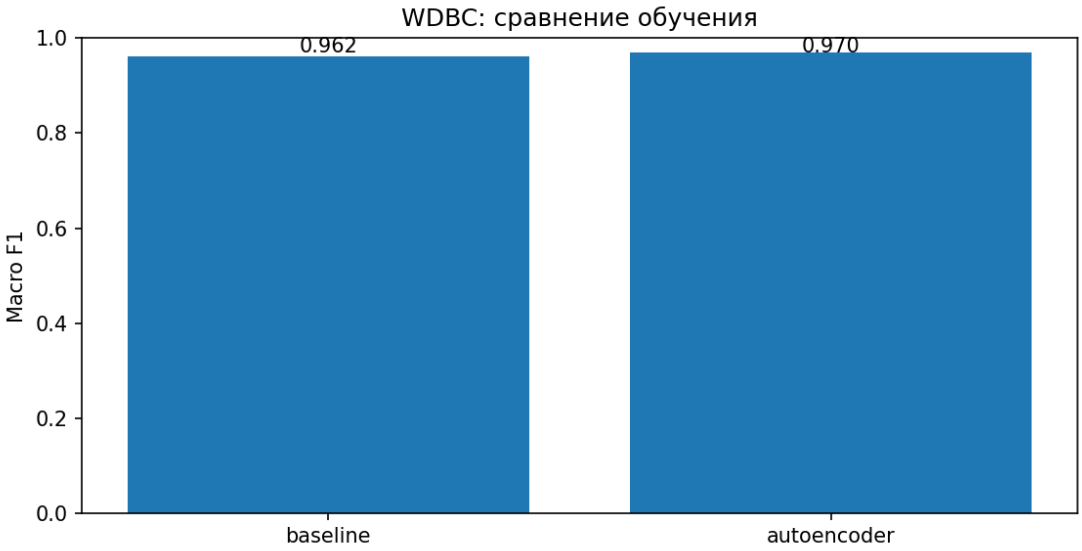
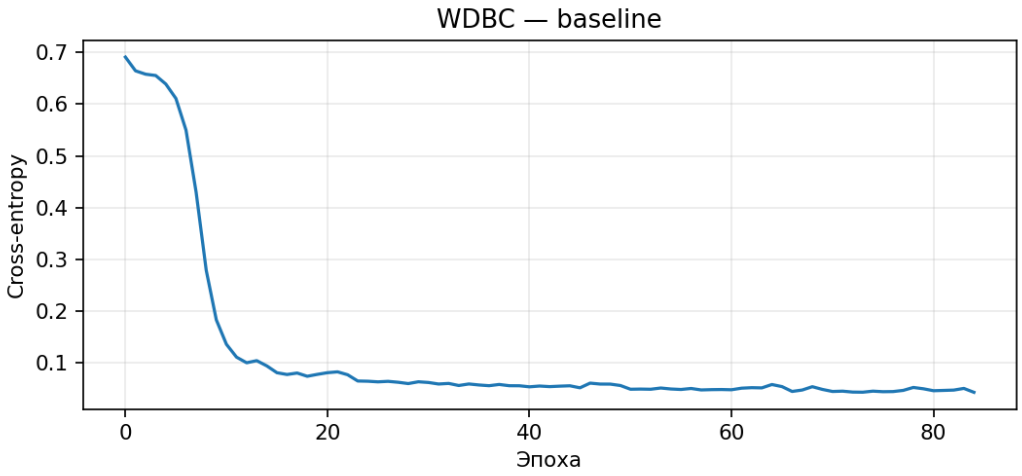
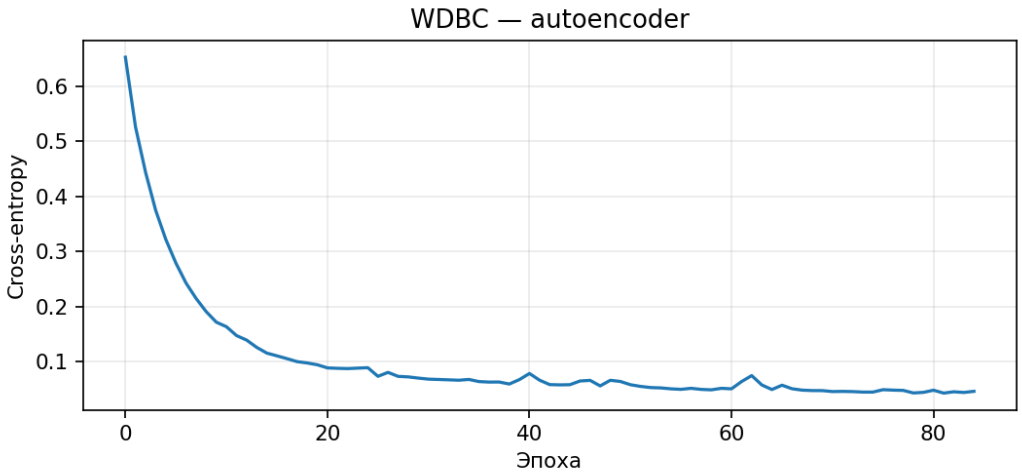


Рисунок — сравнение macro-F1 на WDBC.



Обучение WDBC: baseline, динамика CrossEntropy.

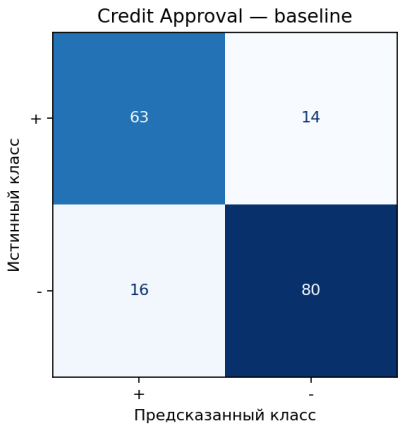


Обучение WDBC: autoencoder, динамика CrossEntropy.

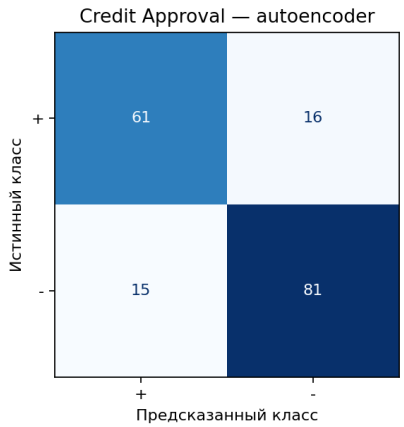
Матрицы ошибок классификации (Confusion Matrix)

Ниже приведены исходные результаты тестовой выборки: строки соответствуют истинным меткам, столбцы — предсказанным. На диагонали отображается число верных предсказаний. Для Credit Approval использован порядок классов «+», «−»; для WDBC — B (доброкачественная) и M (злокачественная).

Набор данных: Credit Approval



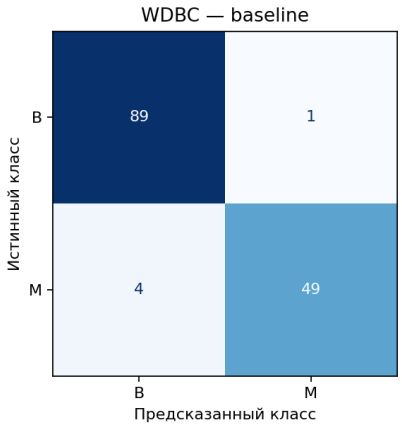
Credit Approval: baseline



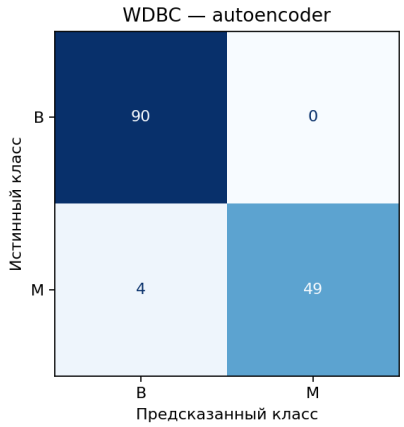
Credit Approval: autoencoder

baseline: правильных 143/173, ошибок 30; autoencoder: правильных 142/173, ошибок 31.

Набор данных: WDBC



WDBC: baseline



WDBC: autoencoder

baseline: правильных 138/143, ошибок 5; autoencoder: правильных 139/143, ошибок 4.

Выводы о проделанной работе

1. Credit Approval: обычное обучение дало macro-F1=0.8249, автоэнкодерное предобучение — 0.8184. Изменение -0.0065.
2. WDBC: обычное обучение дало macro-F1=0.9621, автоэнкодерное предобучение — 0.9695. Изменение +0.0075.
3. Эффект предобучения зависит от датасета, архитектуры и гиперпараметров: улучшение не гарантируется. Выводы сделаны по реально измеренным значениям, без подмены результатов.

4. Для более надёжного статистического сравнения целесообразно повторить обучение на нескольких seed и привести разброс макро-F1.

Приложение А — исходный код ИАД_3.py

Актуальный исполняемый код файла ИАД_3.py; оригинал находится также в src/.

```
"""ЛР 3, вариант 11, Яшков П. А.: baseline vs AE на Credit Approval и WDBC."""
from pathlib import Path
from NN_common import datasets,train,report
ROOT=Path(__file__).resolve().parent
result={}
for name,(X,y,tr,te) in datasets(ROOT).items():
 result[name]=[train(X,y,tr,te,m,ROOT,name) for m in ['baseline','autoencoder']]
report(result,ROOT)
```

Приложение Б — исходный код NN_common.py

Актуальный исполняемый код файла NN_common.py; оригинал находится также в src/.

```
"""Единый эксперимент для ЛР 3 и 4, вариант 11 — Яшков П. А.
Везде один split, одинаковая обработка и одинаковые архитектура/эпохи.
"""
from pathlib import Path
import json,random,time
import numpy as np,pandas as pd,torch
from torch import nn
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, MinMaxScaler,LabelEncoder
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score,f1_score,confusion_matrix,classification_report
from sklearn.neural_network import BernoulliRBM
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
SEED=42
LAYERS=[64,32,16]
EPOCHS=85
PRE_EPOCHS=18
RBM_EPOCHS=14


def seeds():
 random.seed(SEED);np.random.seed(SEED);torch.manual_seed(SEED);torch.set_num_threads(2)


def datasets(root):
 out={}
 for name,filename,target,ignore in [
  ('Credit Approval','credit_approval.csv','Approval',[]),
  ('WDBC','wdbc.csv','Diagnosis',['ID'])]:
  df=pd.read_csv(root/'data'/filename,na_values=['?',''])
  y=LabelEncoder().fit_transform(df[target].astype(str))
  features=df.drop(columns=[target]+ignore)
  tr,te=train_test_split(np.arange(len(df)),test_size=.25,random_state=SEED,stratify=y)
  num=features.select_dtypes(include=np.number).columns.tolist()
  cat=features.select_dtypes(exclude=np.number).columns.tolist()
  trans=[]
  if num:trans.append(('num',Pipeline([('imputer',SimpleImputer(strategy='median'))]),num))
  if
cat:trans.append(('cat',Pipeline([('imputer',SimpleImputer(strategy='most_frequent')),('ohe',OneHotEncoder(handl
e_unknown='ignore',sparse_output=False))]),cat))
  pipe=ColumnTransformer(trans,sparse_threshold=0)
  fit=pipe.fit_transform(features.iloc[tr]);full=pipe.transform(features)
  # Для Bernoulli RBM и всего сравнения каждый признак в диапазоне [0,1]; подгонка только по train.
  scale=MinMaxScaler(clip=True).fit(fit)
  X=scale.transform(full).astype('float32')
  out[name]=(X,y,tr,te)
 return out


class DNN(nn.Module):
 def __init__(self,n_in,n_classes):
  super().__init__()
  nodes=[n_in]+LAYERS
  elems=[]
  for i in range(len(nodes)-1):elems.extend([nn.Linear(nodes[i],nodes[i+1]),nn.Sigmoid()])
  elems.append(nn.Linear(nodes[-1],n_classes));self.layers=nn.Sequential(*elems)
 def forward(self,x):return self.layers(x)


def ae_pretrain(model,X,tr):
 z=torch.tensor(X[tr],dtype=torch.float32)
 for position,n_hidden in zip([0,2,4],LAYERS):
  encoder=nn.Linear(z.shape[1],n_hidden)
  decoder=nn.Linear(n_hidden,z.shape[1])
  auto=nn.Sequential(encoder,nn.Sigmoid(),decoder,nn.Sigmoid())
  opt=torch.optim.Adam(auto.parameters(),lr=.008)
  for _ in range(PRE_EPOCHS):
   for mb in z.split(128):
    opt.zero_grad();loss=nn.functional.mse_loss(auto(mb),mb);loss.backward();opt.step()
  with torch.no_grad():
```

```
   layer=model.layers[position];layer.weight.copy_(encoder.weight);layer.bias.copy_(encoder.bias)
   z=torch.sigmoid(encoder(z)).detach()


def rbm_pretrain(model,X,tr):
 z=X[tr].astype('float64')
 for position,n_hidden in zip([0,2,4],LAYERS):

rbm=BernoulliRBM(n_components=n_hidden,learning_rate=.035,batch_size=32,n_iter=RBM_EPOCHS,random_state=SEED,verb
ose=False)
  rbm.fit(z)
  with torch.no_grad():
   layer=model.layers[position]
   layer.weight.copy_(torch.tensor(rbm.components_,dtype=torch.float32))
   layer.bias.copy_(torch.tensor(rbm.intercept_hidden_,dtype=torch.float32))
  z=rbm.transform(z)


def train(X,y,tr,te,method,root,label):
 seeds()
 model=DNN(X.shape[1],len(np.unique(y)))
 pre_time=time.perf_counter()
 if method=='autoencoder':ae_pretrain(model,X,tr)
 elif method=='RBM':rbm_pretrain(model,X,tr)
 pre_seconds=round(time.perf_counter()-pre_time,3) if method!='baseline' else 0.0
 opt=torch.optim.Adam(model.parameters(),lr=.008)
 tx=torch.tensor(X[tr]);ty=torch.tensor(y[tr],dtype=torch.long)
 losses=[]; start=time.perf_counter()
 for ep in range(EPOCHS):
  permutation=torch.randperm(len(tr));total=0.0
  for ids in permutation.split(64):

opt.zero_grad();z=model(tx[ids]);loss=nn.functional.cross_entropy(z,ty[ids]);loss.backward();opt.step();total+=f
loat(loss.detach())*len(ids)
  losses.append(total/len(tr))
 with torch.no_grad():pred=model(torch.tensor(X[te])).argmax(dim=1).numpy()
 slug='credit' if label.startswith('Credit') else 'wdbc'
 out=root/'results';out.mkdir(exist_ok=True)
 torch.save(model.state_dict(),out/f'{slug}_{method}.pt')
 fig,ax=plt.subplots(figsize=(7,3.4));ax.plot(losses);ax.set(xlabel='Эпоха',ylabel='Cross-
entropy',title=f'{label} — {method}');ax.grid(alpha=.25)
 fig.tight_layout();fig.savefig(out/f'{slug}_{method}_loss.png',dpi=155);plt.close(fig)
 cm=confusion_matrix(y[te],pred)

record={'dataset':label,'method':method,'samples_total':len(X),'features_after_encoding':X.shape[1],'train':len(
tr),'test':len(te),

'architecture':[X.shape[1]]+LAYERS+[len(np.unique(y))],'train_epochs':EPOCHS,'pretrain_epochs':PRE_EPOCHS if
method=='autoencoder' else (RBM_EPOCHS if method=='RBM' else 0),
         'pretrain_seconds':pre_seconds,'train_seconds':round(time.perf_counter()-start,3),
         'accuracy':float(accuracy_score(y[te],pred)),'f1_macro':float(f1_score(y[te],pred,average='macro')),
         'f1_weighted':float(f1_score(y[te],pred,average='weighted')),'confusion_matrix':cm.tolist(),
         'classification_report':classification_report(y[te],pred,zero_division=0),
         'last_cross_entropy':round(losses[-1],6),'loss_history':losses}
 print(f'{label} | {method}: accuracy={record["accuracy"]:.4f}, macro-F1={record["f1_macro"]:.4f} (pretrain
{pre_seconds}s)')
 return record


def report(res,root):
 p=root/'results'/'metrics.json';p.write_text(json.dumps({'variant':11,'student':'Яшков П.
А.','datasets':res,'seed':SEED},ensure_ascii=False,indent=2),encoding='utf-8')
 for name,arr in res.items():
  fig,ax=plt.subplots(figsize=(7.6,4));methods=[x['method'] for x in arr];vals=[x['f1_macro'] for x in arr]
  bars=ax.bar(methods,vals);ax.set_ylim(0,1);ax.set_ylabel('Macro F1');ax.set_title(f'{name}: сравнение
обучения')
  for b,v in zip(bars,vals):ax.text(b.get_x()+b.get_width()/2,min(.97,v+.02),f'{v:.3f}',ha='center')
  fig.tight_layout();fig.savefig(root/'results'/f'compare_{"credit" if name.startswith("Credit") else
"wdbc"}.png',dpi=150);plt.close(fig)
  slug='credit' if name.startswith('Credit') else 'wdbc'
  display_labels=['+', '-'] if slug=='credit' else ['B (benign)', 'M (malignant)']
  for item in arr:
   cm=np.asarray(item['confusion_matrix'],dtype=int)
   fig,ax=plt.subplots(figsize=(5.1,4.0))
   from sklearn.metrics import ConfusionMatrixDisplay
   ConfusionMatrixDisplay(confusion_matrix=cm,display_labels=display_labels).plot(
       ax=ax,cmap='Blues',values_format='d',colorbar=False)
   ax.set_title(f'{name} — {item["method"]}');ax.set_xlabel('Предсказанный класс')
   ax.set_ylabel('Истинный класс');fig.tight_layout()
   fig.savefig(root/'results'/f'confusion_{slug}_{item["method"]}.png',dpi=160)
```

```
plt.close(fig)
```